Network Science

Lecture 02: Graph Theory

Prof. Dr. Michael Granitzer Dr. Kanishka Ghosh Dastidar

Dr. Jelena Mitrovic

Speaker notes

Graph theory serves as the formal foundation for network science. While our intuitive understanding of networks comes from daily experiences with social or technical systems, formal graph theory provides the precise mathematical language needed to analyze structures, optimize paths, and detect patterns. Today's session transitions from high-level concepts to rigorous definitions, focusing on how modeling choices—such as whether edges are directed or weighted—fundamentally change the analytical landscape.

From Networks to Graphs

Last time we saw that networks are everywhere: social, informational, economic, technical. Today we build the formal language to describe them precisely.

Today

- what properties do networks have?
- how do we formalize networks as graphs?
- paths and distance
- breadth-first search
- connectivity and components

Properties of Networks

Guiding Question: What properties does a network have? What do we need to describe it?

Speaker notes

The transition from intuitive "networks" to formal "graphs" begins by identifying the core properties of relational data. We must consider not only the number of entities and connections but also their directionality, strength, and how they evolve over time. Each of these descriptive needs directly corresponds to a specific mathematical construct in graph theory, such as directed edges, weighted functions, or temporal graphs.

What Does a Node Mean?

- In a social network: a **person**
- In the Web: a **page**
- In a financial network: an **institution**
- In a transport network: a **city** or **station**

A node is whatever entity you choose to model. That choice is never neutral — it determines what you can see and what you miss.

Speaker notes

Defining a "node" is a non-trivial modeling decision. The granularity of the model (e.g., modeling departments instead of individuals, or paragraphs instead of whole web pages) depends entirely on the research question. What counts as a node in one analysis might be considered internal structure in another; thus, the choice of node definition is the first step in determining what properties of the system can be observed.

What Does an Edge Mean?

- Friendship: **symmetric** — if A is friends with B, B is friends with A
- Following on Twitter/X: **asymmetric** — A follows B does not mean B follows A
- Email: **directed** — sender and receiver are distinct roles
- Co-authorship: **symmetric** — both contributed to the same paper

An edge encodes a relationship. But relationships differ:

- symmetric vs. directed
- strong vs. weak
- binary vs. weighted.

Speaker notes

The interpretation of an "edge" must align with the nature of the relationship. Symmetric relations (e.g., co-authorship) are modeled with undirected edges, while asymmetric ones (e.g., following on a social platform) require directed edges. This choice affects fundamental properties of the graph, such as reachability and flow, and must be justified by the real-world interaction being captured.

What About Time?

- Networks **grow**: new people join, new links form
- Networks **shrink**: people leave, links break
- Networks **change**: relationships strengthen or weaken
- The **order** of events matters: who connected to whom first?

Most real networks are dynamic. A static graph is a snapshot — useful, but incomplete.

Speaker notes

While static graphs are the standard starting point for analysis, real-world networks are inherently dynamic—growing, shrinking, and evolving over time. Recognizing a graph as a "snapshot" highlight that temporal order often determines the final structure (e.g., who connected to whom first). More advanced techniques in temporal network analysis are needed when the timing of relationships is critical to the outcome.

From Properties to Formal Language

We now have a list of properties we need to capture:

Property	Formal concept
Entities	Nodes (vertices)
Relationships	Edges (links)
Directionality	Directed vs. undirected graphs
Strength	Weighted vs. unweighted graphs
Structure	Adjacency matrix, degree
Time	Temporal / dynamic graphs

Speaker notes

Graph theory provides a mapping between intuitive system properties and formal mathematical concepts. This lecture serves as a roadmap: moving from entities to vertices, relationships to edges, and strength to weights. This formalization is necessary to move from visual intuition to algorithmic analysis.

Modeling Is a Choice

Before we analyse a graph, we decide how to turn a situation into one.

From reality to graph model



Example: an affiliation setting can become a bipartite graph, or a projection, depending on the question.

Graph theory gives us a **formal language**, but the first step is still **epistemic**: choose the entities, choose the relations, and decide what abstraction is good enough for the question.

Speaker notes

The formalization pipeline highlights the transition from messy reality to a structured graph model. Every step involves an epistemic choice about what is essential and what can be ignored. Whether a situation is modeled as a simple graph, a bipartite network, or a temporal sequence depends entirely on the analytical goal.

Takeaway

Describing a network precisely requires answering:

- What are the nodes? What are the edges?
- Are they directed? Weighted?
- Changing over time?

Graph theory gives us a formal language to answer each of these questions.

Speaker notes

Before diving into formal definitions, it is essential to practice the "modeling mindset"—identifying the core components of any relational system and deciding how they should be represented to enable meaningful analysis.

Quick Check

Pick a real-world network you interact with (e.g. your university's course system, a messaging group, a transport network). Write down:

1. What are the **nodes**?
2. What are the **edges**?
3. Is the relation **directed** or **undirected**?
4. Should edges be **weighted**? If so, what does the weight represent?
5. Does the network **change over time**?

Speaker notes

Practical exercise in establishing a model for everyday systems. Identifying nodes and edges—and deciding on direction and weight—forces us to articulate the assumptions we are making about how the system functions before we ever run an algorithm.

Quick Check — Sample Answer

Example: University course enrollment system

1. **Nodes:** Students and courses (two types → bipartite)
2. **Edges:** "Student X is enrolled in course Y"
3. **Direction:** Directed — enrollment is an action from student to course
4. **Weights:** Could represent credit hours, or grade achieved
5. **Time:** Yes — students enroll and drop courses each semester

This is already a bipartite graph with temporal dynamics.

Speaker notes

Even simple university systems reveal non-trivial modeling challenges, such as bipartite structures (linking students to courses) vs. projections (linking students who attend the same course). These decisions are epistemic—they change what the resulting graph reveals about the underlying social reality.

Graphs as Models

Guiding Question: When is a graph as model good enough — and what gets lost?

Why Graph Theory

We identified properties we need to describe. Now we need precise definitions. Graph theory gives us that language and lets us move from intuition to analysis.

Formal Definition

A graph $G = (V, E)$ consists of:

- a set of **vertices** (nodes) V
- a set of **edges** (links) $E \subseteq V \times V$

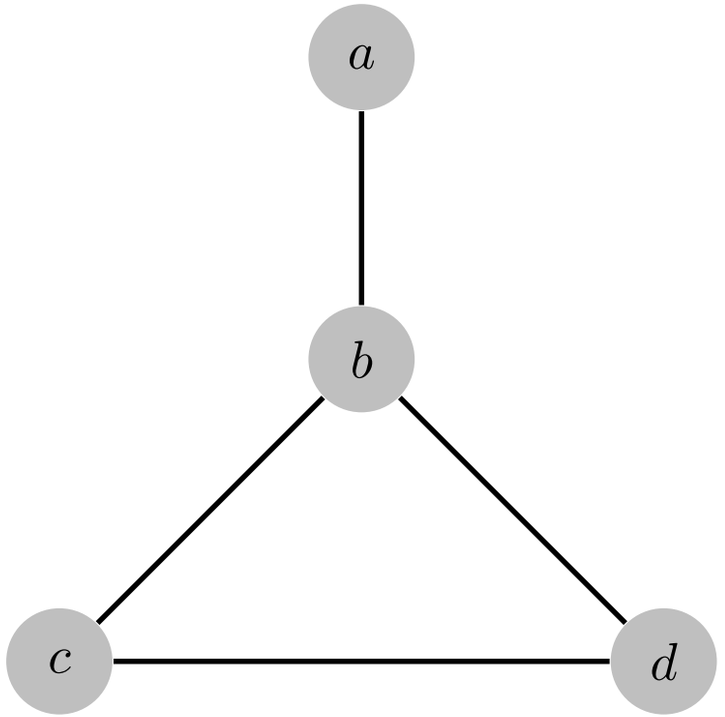
This compact notation already encodes a modeling choice: what counts as a node, what counts as a relationship.

Speaker notes

The formal definition of a graph—a pair (V, E)—is a compact abstraction that strips away domain-specific meanings to focus on pure structure. This mathematical clarity is what allows us to generalize insights across different fields, provided we maintain the connection between the formal model and the real-world interpretation of what nodes and edges represent.

Undirected Graphs

Undirected Graph. An undirected graph treats relationships as symmetric. An edge $\{u, v\}$ connects two vertices without orientation: $\{u, v\} = \{v, u\}$. Source and target are not distinguished.



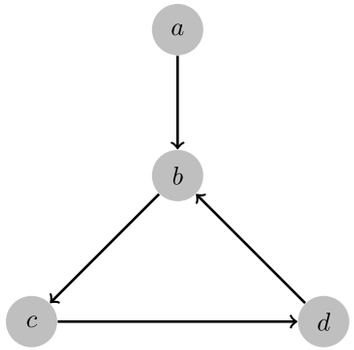
Edges have no arrows, meaning the relation is symmetric (e.g. friendship).

Speaker notes

The undirected graph model makes a strict symmetry assumption: if there is a relationship between u and v, it is bidirectional and identical for both nodes. This is the simplest possible model but is restricted to relations like mutual friendship or physical connectivity where no hierarchy or flow is present.

Directed Graphs

Directed Graph. An edge $(u, v) \in E$ has a strict orientation from u to v. (u, v) does not imply (v, u). Directions matter whenever the relation is asymmetric (e.g. following, citing).



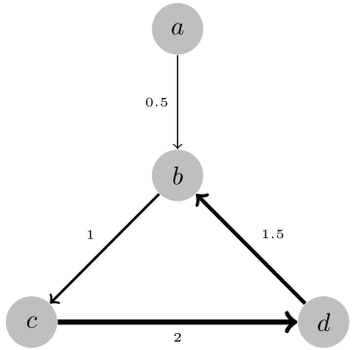
Edges have arrows indicating direction. The same set of nodes can yield very different models depending on this choice.

Speaker notes

Directing edges introduces a fundamental asymmetry into the graph. Directionality is not merely an aesthetic choice; it fundamentally alters the concepts of degree and reachability. In directed models, a path from A to B does not imply a path from B to A, which is critical for representing flow-based systems like information dissemination or financial transactions.

Weighted Graph

Weighted Graph. A graph equipped with a weight function $w : E \to \mathbb{R}$. The weight $w(e)$ lets edges carry strength, cost, length, probability, or capacity.



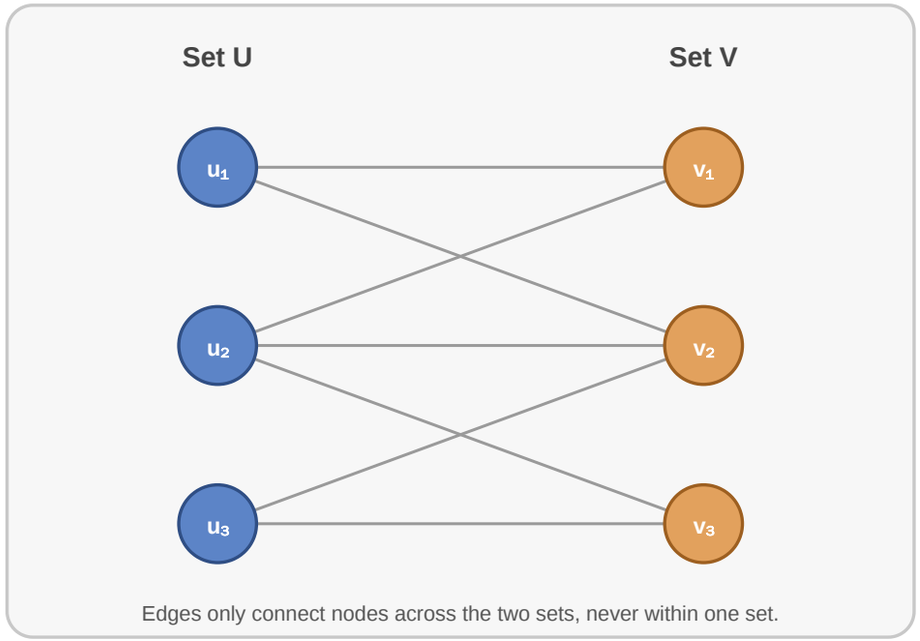
Each edge carries a numerical value. The same topological structure can mean vastly different things depending on what the weights encode.

Speaker notes

Weights add a dimension of continuous intensity to the binary "edge versus no edge" model. In a transport network, weights typically represent costs or distances; in an interaction network, they might represent frequency or duration. The same topological structure can behave very differently under different weighting schemes.

Bipartite Graphs

Bipartite Graph. A graph $G = (U, V, E)$ with two disjoint node sets ($U \cap V = \emptyset$) where all edges connect a node from U to a node from V. Edges never connect two nodes within the same set.



Modelling of different kinds of nodes and limited edges. Useful when tracking relations between two inherently different sets of entities (e.g. students and courses, users and products).

Speaker notes

Bipartite models are specialized for tracking relationships between two disjoint sets of entities where internal connections are forbidden by the system's rules (e.g., students do not "enroll" in other students). This structure is the basis for many recommendation engines and collaborative filtering techniques.

Sparse, Dense, and Random Graphs

Not all graphs differ qualitatively. They also differ quantiatively.

> **Sparse graph.** A graph with relatively few edges compared to the maximum possible number.
>
> **Dense graph.** A graph with many edges, often close to the maximum possible number.
>
> **Random graph.** A graph generated by a probabilistic rule, e.g. each possible edge appears independently with probability p.

- many real social and information networks are **sparse**
- dense graphs are often easier to reason about theoretically, but rarer at scale
- random graphs are important as a **baseline model** for comparison

Speaker notes

This is the first point where graph theory moves beyond "what kind of edge is this?" toward "what kind of structure does the whole graph have?" Sparse and dense graphs differ in algorithmic cost, storage behavior, and typical path lengths. Random graphs matter because they give us a null model: what structure would appear even without any social, technological, or organizational mechanism?

Graph Representations

How would you store (write down) a graph on paper?

Speaker notes

A graph's representation is distinct from its topology. How we store a graph—as an edge list, adjacency list, or adjacency matrix—affects the scalability and performance of the algorithms we run. Matrices enable algebraic insights, while adjacency lists are the standard for efficient graph traversal. Additionally, whether two graphs drawn differently are the same (isomorphism) is a central question in structural analysis.

Graph Representations

How would you store (write down) a graph on paper?

Edge list

```
(u1, v1)
(u1, v2)
(u2, v2)
(u3, v1)
(u3, v2)
```

Compact and easy to exchange,
but neighbor lookup is slower.

Adjacency list

```
u1: v1, v2
u2: v2
u3: v1, v2
v1: u1, u3
v2: u1, u2, u3
```

Best default for traversal:
neighbors are stored directly.

Adjacency matrix

	v1	v2	v3
v1	0	1	1
v2	0	0	1
v3	1	1	1

Rows and columns are node labels.
Fast lookup, but space-heavy.

Common representations are edge lists, adjacency lists, and adjacency matrices. They encode the same graph, but each supports different algorithms and scales differently.

Why Representations Matter

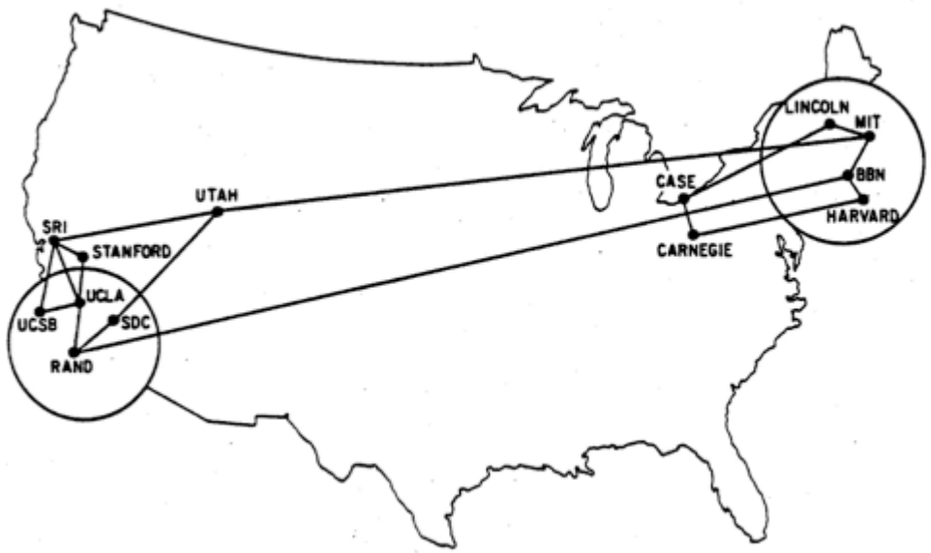
Different representations make different operations easier.

- edge lists are compact, easy to exchange, and natural for tabular data
- adjacency lists are good for traversal because neighbors are stored directly
- adjacency matrices are good for algebraic methods and constant-time edge lookup

Representation	Strength	Limitation
Edge list	Lean, simple, good for import/export	Slow to test whether a specific edge exists
Adjacency list	Best default for BFS/DFS on sparse graphs	Less convenient for matrix-based methods
Adjacency matrix	Fast edge lookup, linear algebra friendly	Uses $O(\

Real-World Example: ARPANET

The ARPANET already shows how quickly a technical system becomes a graph with non-trivial structure (Easley & Kleinberg, 2010).



Adjacency List (1970):

```
SRI:      [UCSB, UCLA, Stanford, UTAH]
UCSB:     [SRI, UCLA]
UCLA:     [SRI, UCSB, RAND, Stanford]
RAND:     [UCLA, BBN, SDC]
SDC:      [RAND, UTAH]
Stanford: [SRI, UCLA]
Utah:     [SRI, SDC, MIT]
Case:     [Carnegie, Lincoln]
Carnegie: [Case, Harvard]
Harvard:  [Carnegie, BBN]
BBN:      [RAND, MIT, Harvard]
MIT:      [UTAH, BBN, Lincoln]
Lincoln:  [Case, MIT]
```

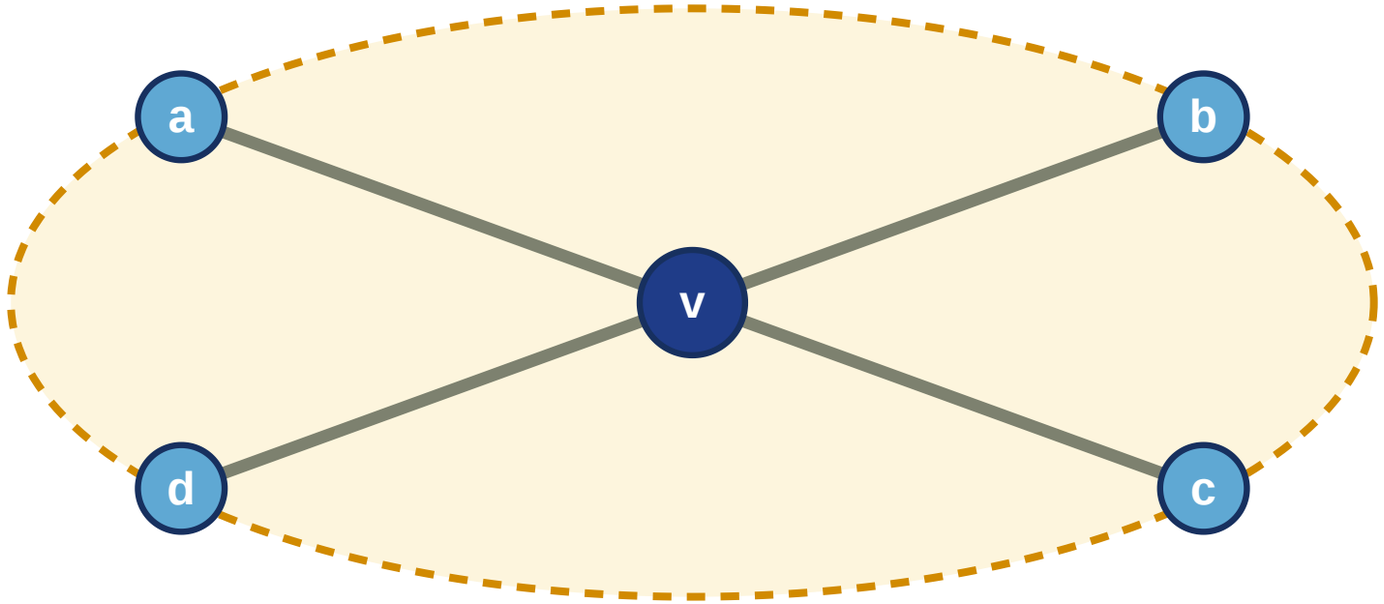
ARPANET topology (1969–1977). *Nodes are universities and research labs, edges are data links. One of the earliest computer networks.*

Neighbourhood and Degree

Neighbourhood. The neighbourhood $N(v)$ of a node v is the set of all nodes connected to v by an edge.

Degree. The degree $\deg(v)$ is the number of edges incident to v. In simple graphs, $\deg(v) = |N(v)|$.

Neighbourhood N(v) = {a, b, c, d}



Degree deg(v) = 4

Speaker notes

The concepts of neighbourhood and degree allow us to quantify local node importance. In directed networks, these bifurcate into in-degree (prestige or attention) and out-degree (influence or activity). Understanding these local metrics is the first step toward more global measures like centrality.

Beyond Simple Graphs

The basic definition $G = (V, E)$ is only the start. In practice, we often need richer graph models.

Graph type	What is added?	Typical use
Labelled graph	names or labels on nodes/edges	named entities, relation types
Typed graph	explicit node and edge classes	users, posts, organizations
Property graph	attributes on nodes and edges	metadata, timestamps, weights
Multigraph	multiple edges between same nodes	repeated interactions, transport links
Temporal graph	time attached to edges or snapshots	evolving communication or contact networks

Key point: the graph formalism is flexible. The "right" graph depends on what information must be preserved for the question.

Speaker notes

This slide broadens the students' notion of what "a graph" can be. Many real analytical systems are not simple unlabeled graphs; they have node types, relation types, timestamps, and attributes. This is only a teaser here, but it prepares the later idea that modeling choices affect which algorithms and interpretations remain valid.

Teaser: Properties We Will Use Later

Once we have a graph, we can ask structural questions beyond "is there an edge?"

- **degree distribution:** how are node degrees distributed across the whole graph?
- **centrality:** which nodes are structurally important or influential?
- **components:** where is the graph connected or fragmented?
- **path lengths:** how many steps separate typical pairs of nodes?

These are only teaser concepts for now. We will return to them in later lectures with formal definitions and applications.

Speaker notes

This slide creates a bridge from basic graph formalization to the rest of the course. Degree distribution moves from local degree to population-level heterogeneity; centrality generalizes "important node" beyond immediate neighborhood. Students do not need the formulas yet, but they should see that graph definitions are the entry point to later network analysis.

Takeaway

> Graph theory is useful because it gives a precise language for the properties we identified, but every graph still reflects a modeling choice that must be interpreted.

Speaker notes

Concluding the formalization module: changing the node or edge definition can lead to entirely different analytical results. Graph theory provides the toolkit, but the researcher provides the critical reasoning that links the formal model to the system's consequences.

Quick Check

Consider a simple 5-node undirected graph: A-B, B-C, C-D, D-A, A-C. Here is an edge list and a function to convert it to an adjacency dictionary, but two key lines are missing. **What goes in the blanks?**

```
edges = [("A","B"), ("B","C"), ("C","D"), ("D","A"), ("A","C")]

def edges_to_adj(edges):
    adj = {}
    for u, v in edges:
        if u not in adj: adj[u] = []
        if v not in adj: adj[v] = []

        adj[u].append(____)
        adj[____].append(____)
    return adj
```



Speaker notes

This exercise reinforces the operational understanding of graph representations. For undirected graphs, students must remember that the adjacency entry must be symmetric—if A is linked to B, then B must also be explicitly linked to A in the representation.

Quick Check — Solution

```
edges = [("A","B"), ("B","C"), ("C","D"), ("D","A"), ("A","C")]

def edges_to_adj(edges):
  adj = {}
  for u, v in edges:
      if u not in adj: adj[u] = []
      if v not in adj: adj[v] = []

      adj[u].append(v)
      # Undirected graph implies symmetric edges
      adj[v].append(u)
  return adj
```

Key insight: Undirected graphs require symmetric entries. If a graph is directed, the second append line is omitted. Why should we not omit the second append line in undirected cases?



Speaker notes

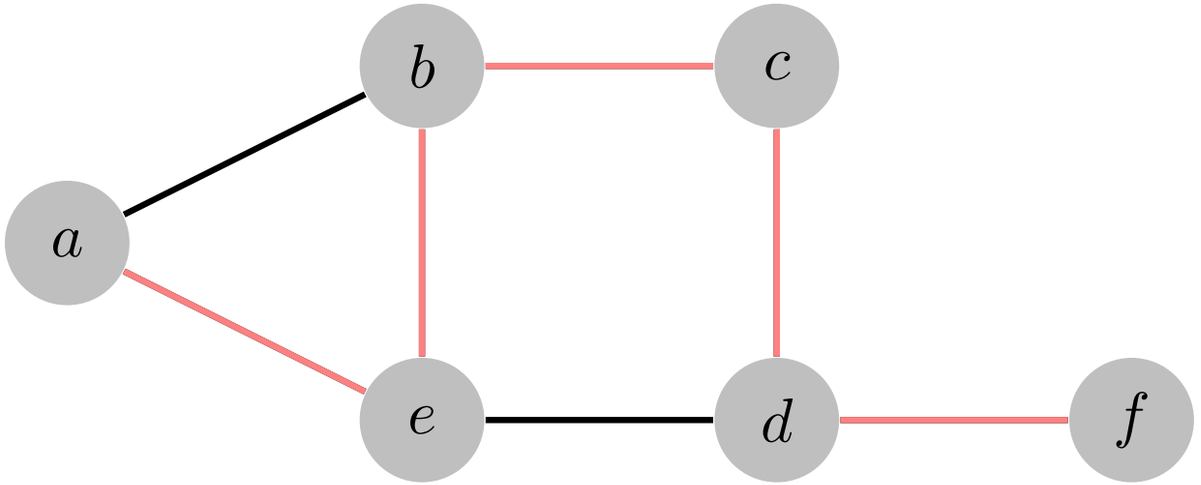
Contrast the symmetry of undirected representations with directed ones. In a directed graph, the second symmetric entry would be omitted, as reachability becomes one-way. This transition is essential for understanding more complex flow models.

Paths and Distance

Guiding Question: What does it mean for one node to "reach" another?

Paths

A path is a sequence of vertices where each consecutive pair is connected by an edge.



> ***A path highlighted in a graph.*** *The colored edges trace a route from a start node to an end node. The path length equals the number of edges traversed.*

Speaker notes

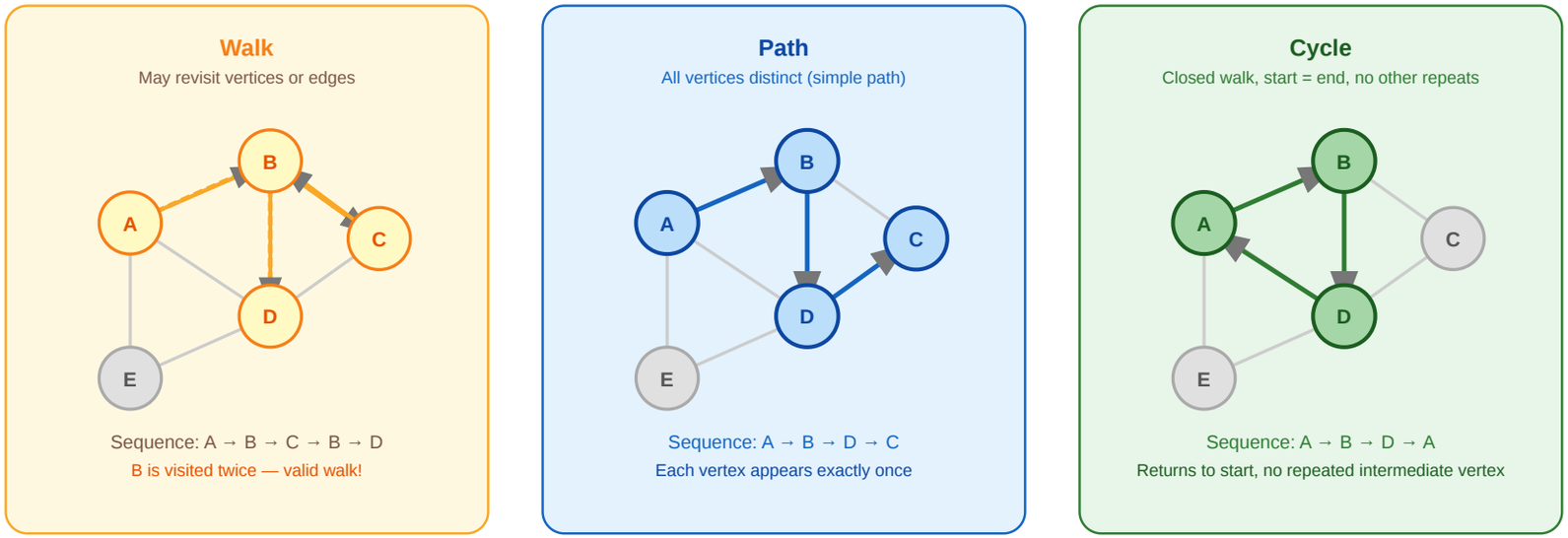
Paths and distance are the fundamental concepts for analyzing connectivity. A path represents a possible route through the system; the length of that route (number of edges) corresponds to the "cost" or "delay" in information dissemination. Tracking these routes allows us to quantify how easily signals can move across the network.

Walk, Path, Cycle

Walk. A walk of length k is a sequence $v_0, v_1, \ldots, v_k$ where $(v_{i-1}, v_i) \in E$ for all i. Vertices may be repeated.

Path. A path is a walk where all vertices are distinct: $v_i \neq v_j$ for $i \neq j$. Also called *simple path*.

Cycle. A cycle is a walk $v_0, v_1, \ldots, v_k$ with $v_0 = v_k$ and all intermediate vertices distinct.



Directed vs. undirected. In an undirected graph, an edge can be traversed in either direction. In a directed graph, each step must follow the edge direction.

Speaker notes

Formalizing the distinction between walks, paths, and cycles is critical for traversal algorithms. While a walk can revisit nodes, a simple path avoids any repetition. Shortest-path algorithms fundamentally work with simple paths, as revisiting a vertex can never shorten the distance between two points in a standard graph.

Path Length, Shortest Path, Diameter

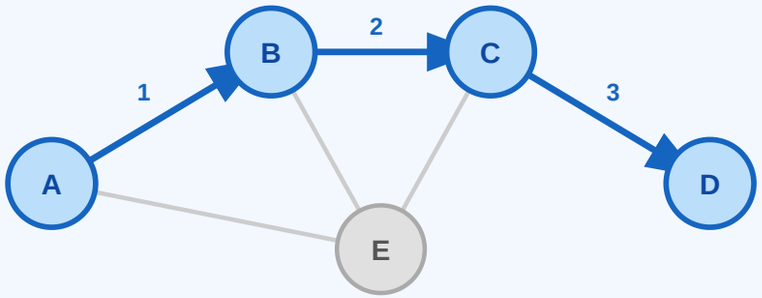
Path length. The length of a path $v_0, \ldots, v_k$ is k (number of edges). In a weighted graph: $\sum_{i=1}^{k} w(v_{i-1}, v_i)$.

Shortest path. $\mathrm{dist}(u, v) = \min\{k \mid \text{path of length } k \text{ from } u \text{ to } v\}$. If no path exists, $\mathrm{dist}(u, v) = \infty$.

Diameter. $\mathrm{diam}(G) = \max_{u,v \in V} \mathrm{dist}(u, v)$ — the longest shortest path in the graph.

Path Length

Number of edges in the path



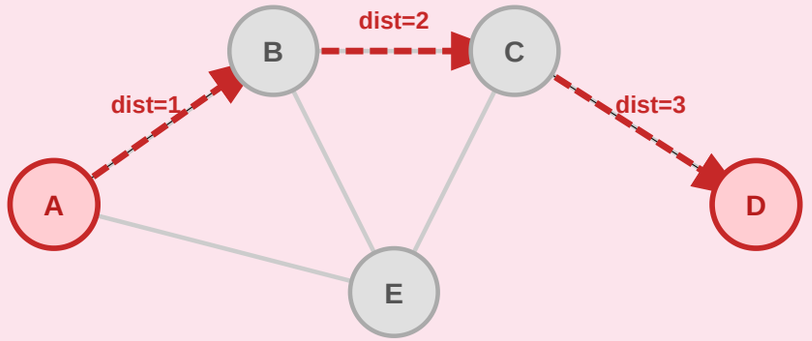
Path A → B → C → D has length 3

(3 edges traversed)

Weighted: length = sum of edge weights

Diameter

Longest shortest path in the graph



diam(G) = dist(A, D) = 3

The farthest pair determines the diameter

In directed graphs, dist(u,v) ≠ dist(v,u) in general

Speaker notes

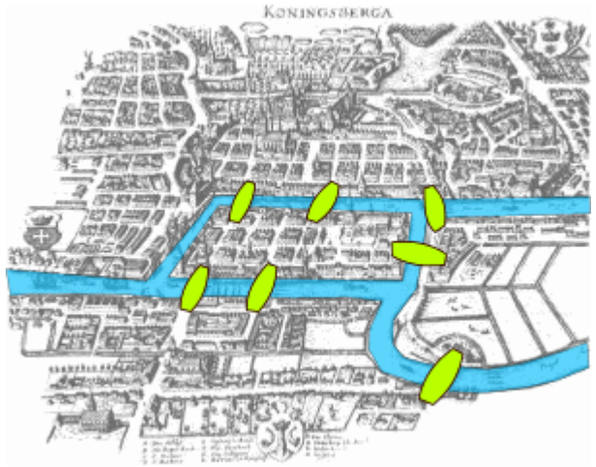
In undirected graphs, distance is symmetric. In directed graphs, the shortest path from u to v may differ from v to u. In weighted graphs, path length is the sum of weights, not edge count.

Why Paths Matter

- they formalize reachability
- they give us a notion of distance
- they underpin routing, diffusion, and traversal algorithms

A Classical Motivation

The Konigsberg bridges problem is one of the classic entry points into graph theory.



> ***The Seven Bridges of Konigsberg (1736).*** *Euler asked whether one could walk through the city crossing each bridge exactly once. He proved it impossible — and in doing so, founded graph theory.*

Speaker notes

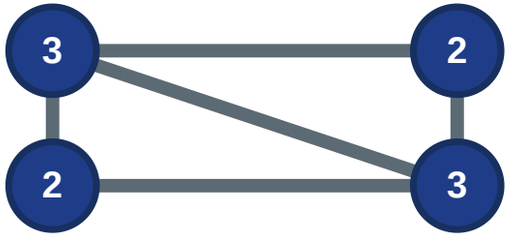
Leonhard Euler's solution to the Seven Bridges of Königsberg (1736) is considered the founding moment of graph theory. Euler's key insight was that the physical layout, distances, and shapes of the island and river banks were irrelevant; only the pattern of connections (the topology) determined the solution. This abstraction of physical reality into nodes and edges allowed for a rigorous proof of what is—and is not—possible in any networked system.

Eulerian Path and Circuit

Eulerian Path. A walk traversing every edge exactly once. Exists iff exactly 0 or 2 vertices have an odd degree.

Eulerian Path

Exactly two odd-degree vertices



Odd degrees: 2 endpoints

Eulerian Circuit. An Eulerian path that starts and ends on the same vertex. Exists iff *every* vertex has an even degree.

Eulerian Circuit

All vertices have even degree



Closed walk possible

Eulerian Path and Circuit

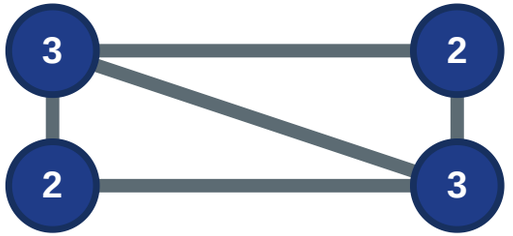
Eulerian Path. A walk traversing every edge exactly once. Exists iff exactly 0 or 2 vertices have an odd degree.

Intuitive Proof: Think of rooms with doors (=edges). If a room has an even number of doors, you can enter and leave it an equal number of times. If it has an odd number of doors, you must start or end in that room.

Eulerian Circuit. An Eulerian path that starts and ends on the same vertex. Exists iff *every* vertex has an even degree.

Eulerian Path

Exactly two odd-degree vertices



Odd degrees: 2 endpoints

Eulerian Circuit

All vertices have even degree



Closed walk possible

Eulerian Path and Circuit

Eulerian Path. A walk traversing every edge exactly once. Exists iff exactly 0 or 2 vertices have an odd degree.

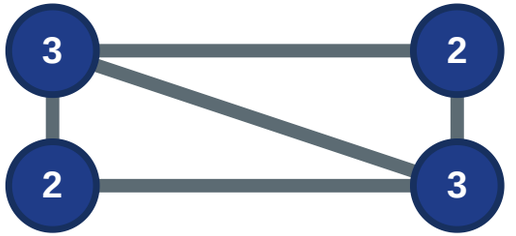
Intuitive Proof: Think of rooms with doors (=edges). If a room has an even number of doors, you can enter and leave it an equal number of times. If it has an odd number of doors, you must start or end in that room.

Eulerian Circuit. An Eulerian path that starts and ends on the same vertex. Exists iff *every* vertex has an even degree.

Intuitive Proof: If there are no rooms with an odd number of doors, you can only start and end at the same room.

Eulerian Path

Exactly two odd-degree vertices



Odd degrees: 2 endpoints

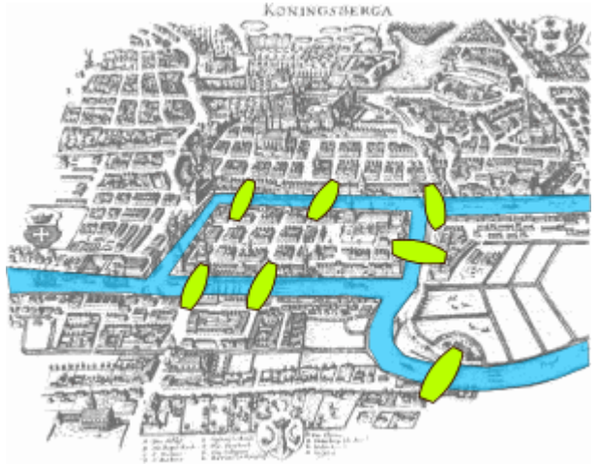
Eulerian Circuit

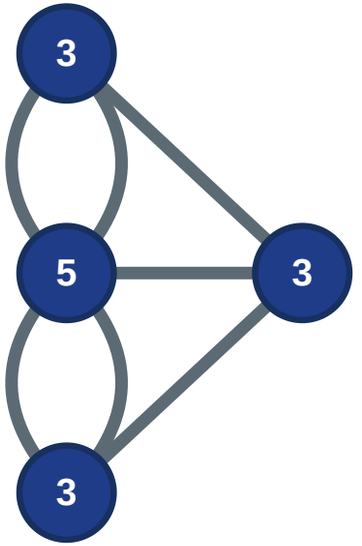
All vertices have even degree



Closed walk possible

The Königsberg Solution





Connecting back to Königsberg: all four land masses had odd degrees (3, 3, 3, 5), violating both conditions.

Takeaway

> Paths turn connectivity into something analyzable: reachability, distance, redundancy, and traversal constraints.

Speaker notes

The Königsberg bridge problem illustrates the power of structural reasoning. By identifying that an Eulerian path requires a limited number of odd-degree vertices, Euler moved from exhaustive trial-and-error to a universal structural rule. In practical terms, these concepts underpin routing problems where every link (road, data line, or social contact) must be utilized exactly once.

Quick Check

Consider a 5-node graph: `adj = {1: [2, 3], 2: [1, 4], 3: [1, 4], 4: [2, 3, 5], 5: [4]}`

Here is a recursive function to find *any* path between two nodes, but the crucial recursive step is missing. **What goes in the blank?**

```
def find_path(graph, start, end, path=[]):
    path = path + [start]
    if start == end: return path
    for node in graph.get(start, []):
        if node not in path:
            # How do we continue the search?
            result = _________________________________
            if result is not None: return result
    return None
```



Speaker notes

This recursive pathfinding exercise introduces the concept of backtracking. The function explores one branch to its end, and if the target is not found, "unwinds" to try the next available neighbor. This "deep" exploration strategy is the foundation of Depth-First Search.

Quick Check — Solution

The algorithm is called depth first search (DFS) - we traverse every edge as soon as we encounter it.

```python
def find_path(graph, start, end, path=[]):
    path = path + [start]
    if start == end: return path
    for node in graph.get(start, []):
        if node not in path:
            # Recursive call on the neighbor
            result = find_path(graph, node, end, path)
            if result is not None: return result
    return None
```

Key insight: DFS explores deep first and finds *any* path. How would we change the approach to guarantee finding the *shortest* path?

Speaker notes

Depth-First Search (DFS) is efficient for exploring reachability but does not guarantee the discovery of the most efficient route. Because it follows a single path as far as possible, it may return a very long path even when a much shorter one exists. For shortest-path guarantees, we must shift to the breadth-first strategy.

Graph Search Algorithms

Guiding Question: How can we systematically find a node — or the shortest route to it — in a large graph?

Breadth-First Search

Breadth-first search explores a graph layer by layer, guaranteeing shortest paths in unweighted graphs.

Algorithm: BFS

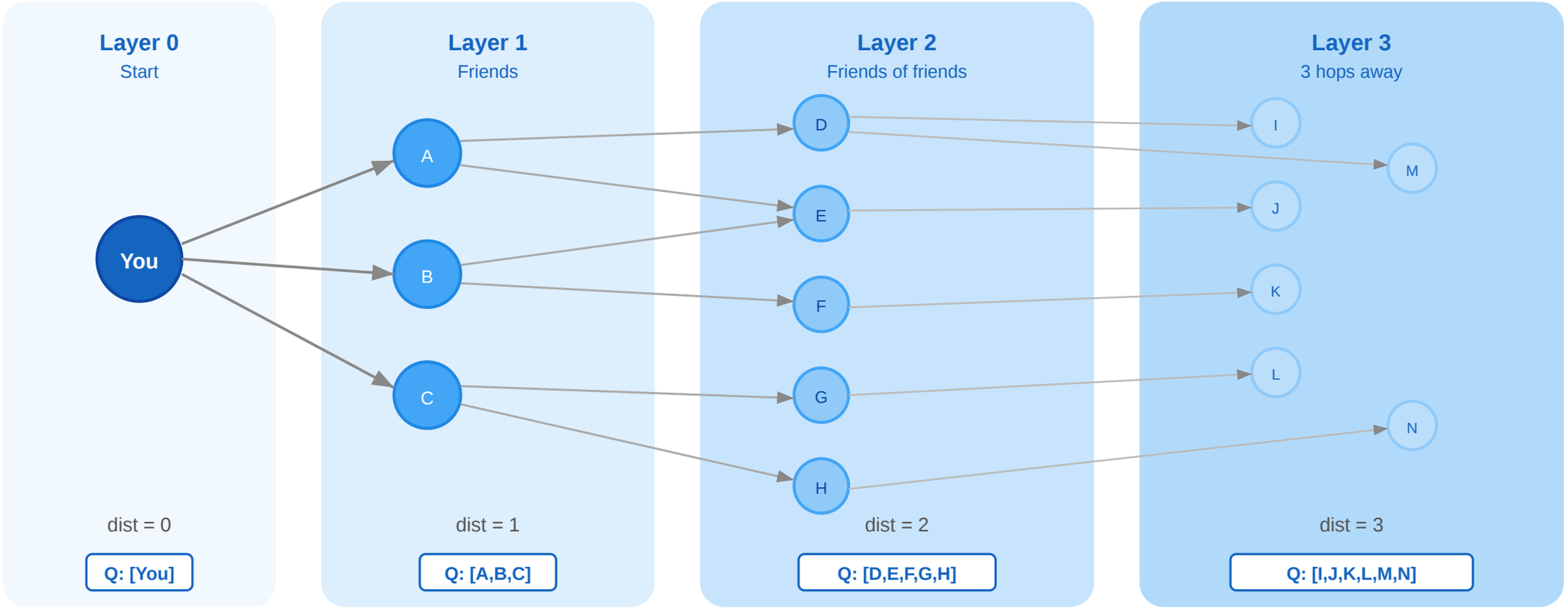
(Input: Graph $G = (V, E)$, source node s)

1. Initialize a **queue** Q and add s
2. Mark s as **visited**
3. While Q is not empty:
   - Extract current node u from the front of Q
   - For each neighbor v of u: if v is **not visited**, mark v as visited and add v to the back of Q

Key Properties:

- Uses a **FIFO** (First-In, First-Out) queue.
- Discovers nodes strictly layer by layer.
- Runs in $O(|V| + |E|)$ time.
- Useful for shortest paths, components, and reachability.

Visual Intuition

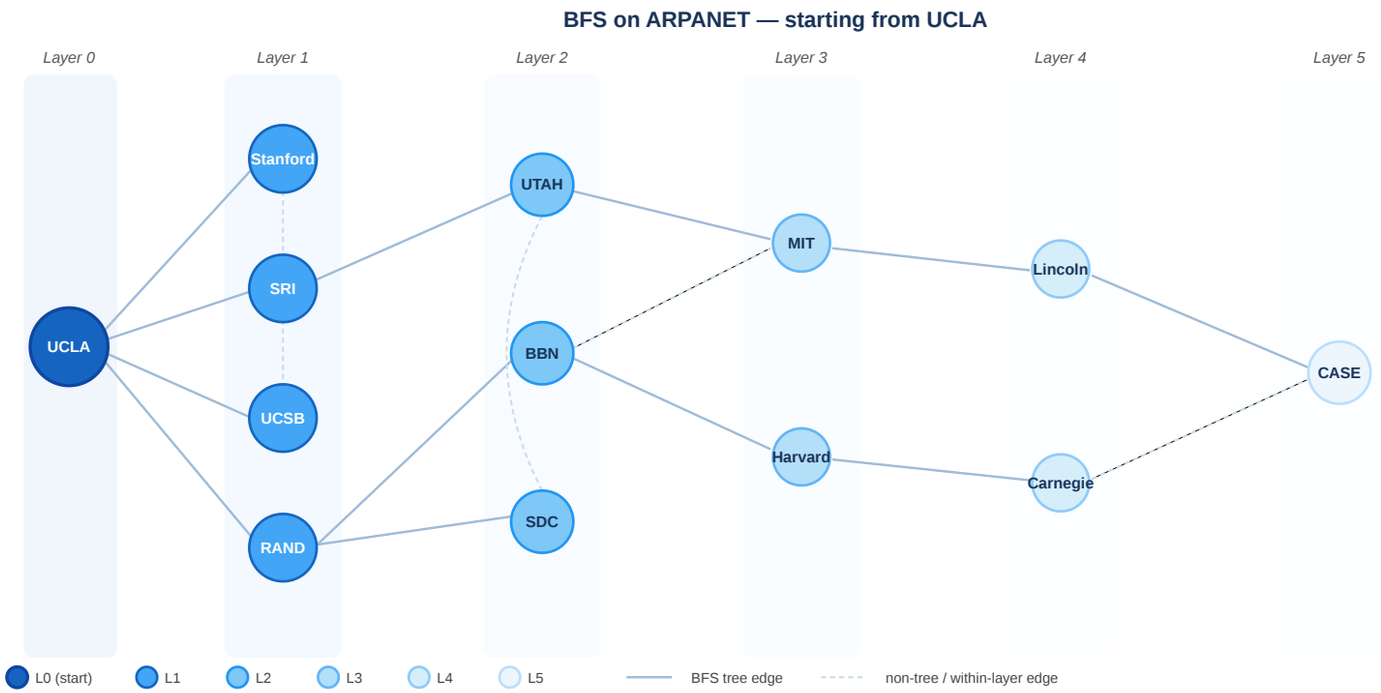


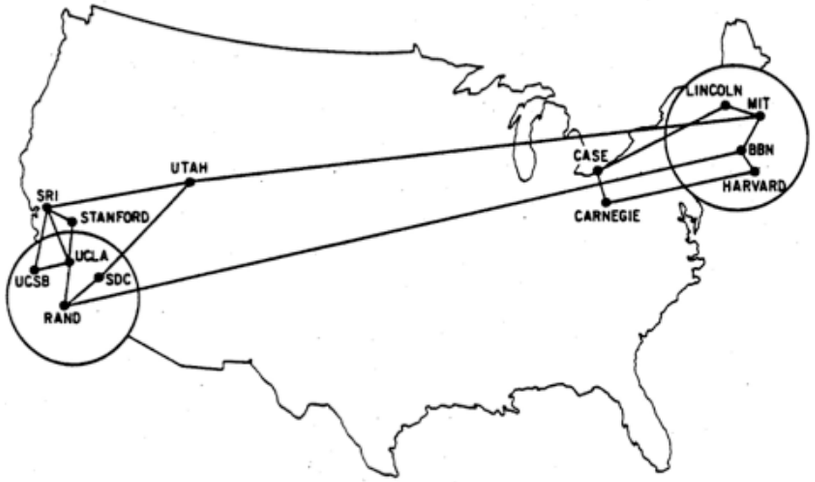
BFS layer-by-layer expansion. Starting from "You" (layer 0), BFS discovers all direct neighbors (layer 1), then all not-yet-visited neighbors of those (layer 2), and so on.

Speaker notes

The core of Breadth-First Search (BFS) is the FIFO (First-In, First-Out) queue. By processing nodes in the exact order they are discovered, the algorithm creates a "ripple" effect that expands layer by layer. This ensures that when a node is first encountered, the path taken is the shortest possible in an unweighted graph, as no shorter path could have remained unvisited in the previous layers.

BFS on the ARPANET





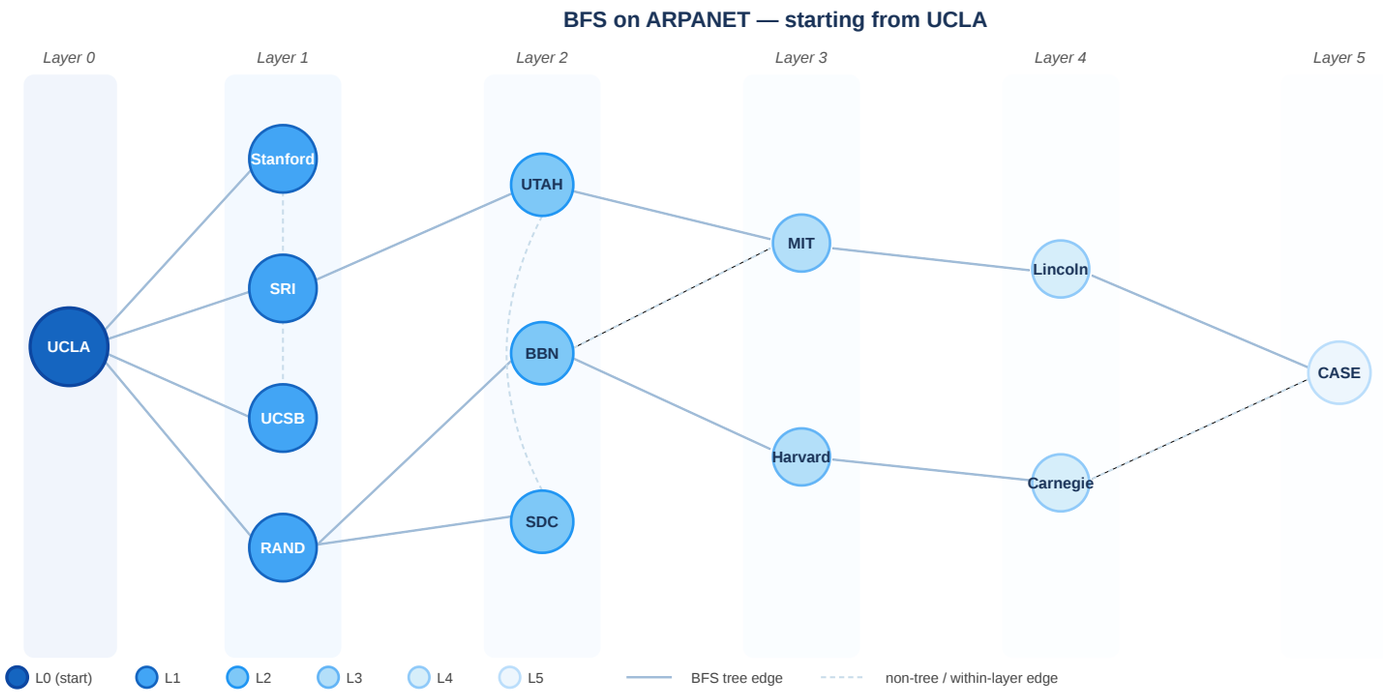
BFS applied to the ARPANET topology starting from UCLA. Node color encodes BFS layer (distance).

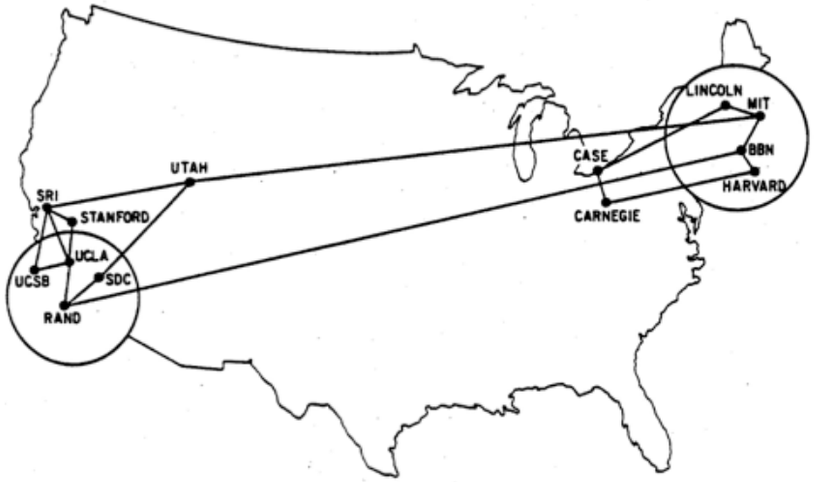
Discussion: The BFS visualization shows some nodes with multiple incoming edges from the previous layer. Is this possible in a strict BFS spanning tree?

Speaker notes

Applying BFS to the ARPANET demonstrates its role as a fundamental routing primitive. While the physical topology is fixed, the BFS layers project a logical "distance graph" onto the network. Note the distinction between the all-possible level graph and a single BFS spanning tree, which represents one possible set of optimal routes from the source.

BFS on the ARPANET





BFS applied to the ARPANET topology starting from UCLA. Node color encodes BFS layer (distance).

Discussion: The BFS visualization shows some nodes with multiple incoming edges from the previous layer. Is this possible in a strict BFS spanning tree?

Answer: No. A true BFS tree has exactly one parent per node. Drawing *all* shortest-path edges creates a "level graph", not a spanning tree!

Complexity

- Queue-based BFS runs in $O(|V| + |E|)$ — each vertex and each edge is visited at most once
- This makes BFS the standard exact method for shortest paths in **unweighted** graphs

Directed graphs: BFS works identically — it follows edges in their given direction.

Weighted graphs: BFS does **not** give shortest paths when edges have different weights. For weighted shortest paths, use Dijkstra's algorithm.

Speaker notes

BFS achieve a linear time complexity of $O(|V| + |E|)$, visitng every reachable node and edge exactly once. This is the theoretical optimum for any algorithm that must "read" the entire graph. However, this efficiency relies on the unweighted assumption; if edges have varying costs, the BFS "layer" guarantee no longer holds, as a multi-hop high-cost path may be more expensive than a single-hop low-cost one.

Depth-First Search

Depth-first search traverses as deep as possible along each branch before backtracking.

Algorithm: DFS

(Input: Graph $G = (V, E)$, source node s)

1. Initialize a **stack** S and add s
2. While S is not empty:

- Pop current node u from the top of S
- If u is **not visited**:
  - Mark u as **visited**
  - Push all neighbors of u onto the stack S

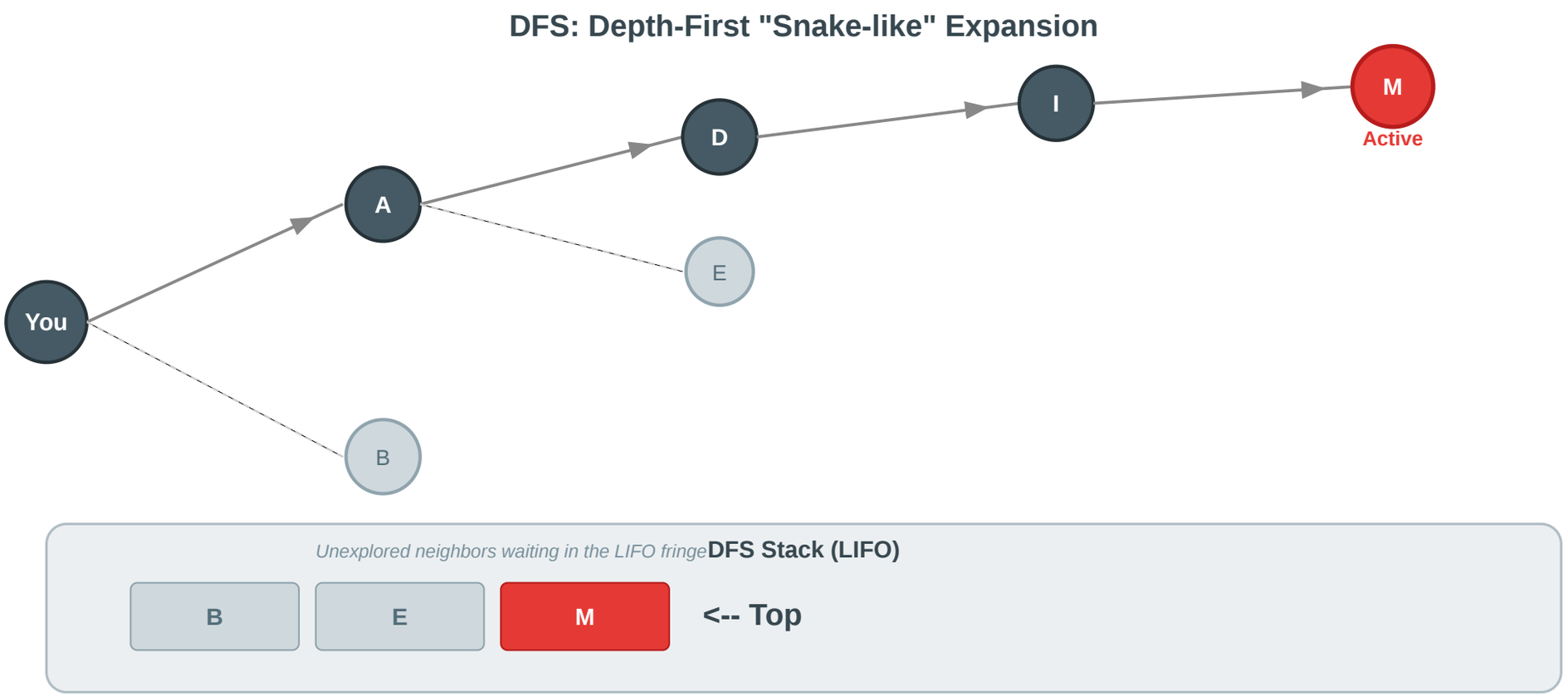
Key Properties:

- Uses a **LIFO** (Last-In, First-Out) stack.
- Explores deep before wide.
- Runs in $O(|V| + |E|)$ time.
- Standard for finding cycles and topological sorting.

Speaker notes

DFS is the stack-based counterpart to BFS. Instead of exploring layer-by-layer, it follows a single path to its end, pushing newly discovered neighbors onto a stack (LIFO). This means the most recently discovered node is always explored next, creating a deep, "snake-like" search pattern. Note that the order of neighbors pushed determine the exact path, but the general deep behavior is invariant.

DFS Visual Intuition



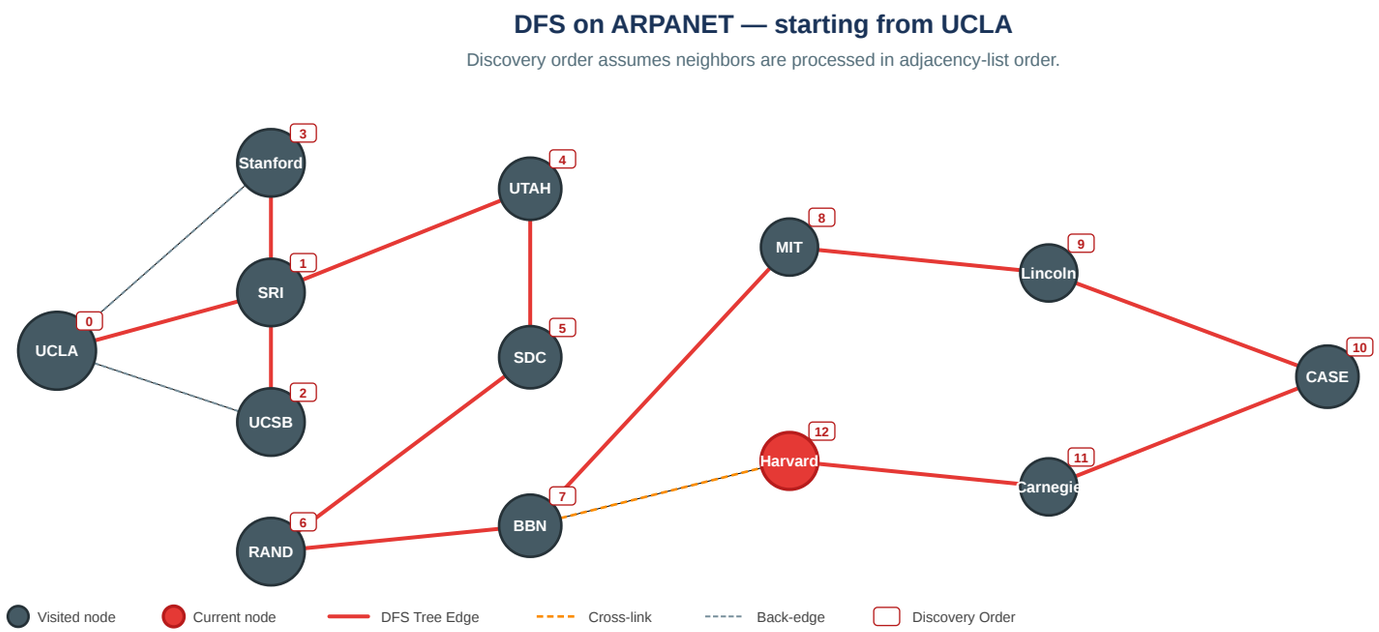
DFS follows one path as deep as possible before backtracking to the last node with unexplored neighbors.

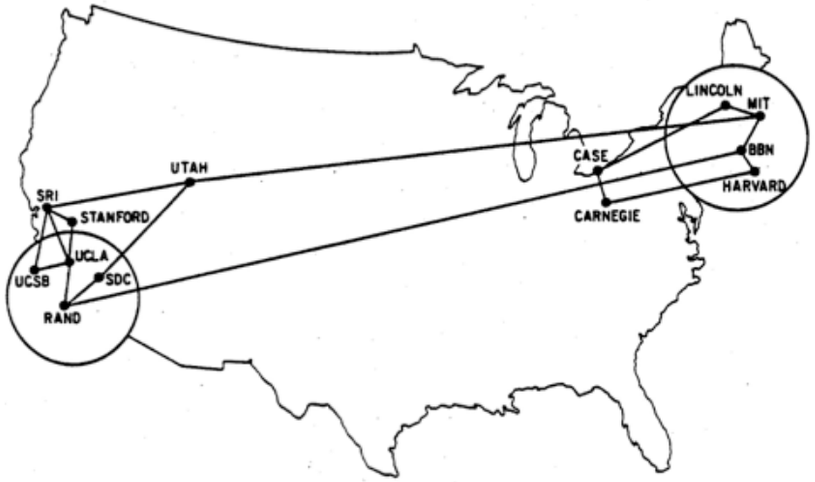
DFS deep expansion. Starting from "You", DFS picks one neighbor and immediately proceeds to that neighbor's neighbors. It only returns to explore other branches (backtracking) when it hits a "dead end" where all neighbors of the current node are already visited.

Speaker notes

In contrast to the "ripple in a pond" behavior of BFS, DFS acts like a "snake in a maze". It creates a long, narrow spanning tree. While BFS is optimal for shortest paths, DFS is often faster for simply checking if any path exists or for exploring deep into a dense graph.

DFS on the ARPANET





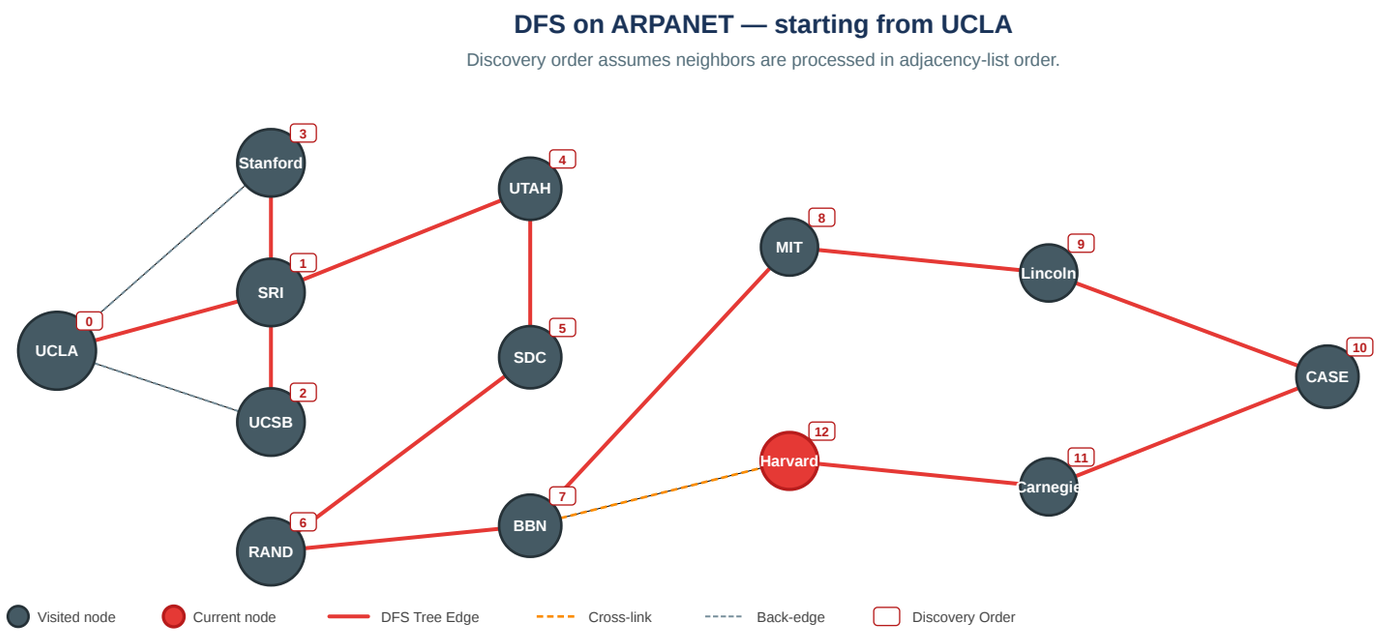
DFS applied to the ARPANET. Node numbers indicate the discovery order if neighbors are processed in the listed adjacency-list order. Notice how the search can wander far across the network before exploring nodes physically close to the start.

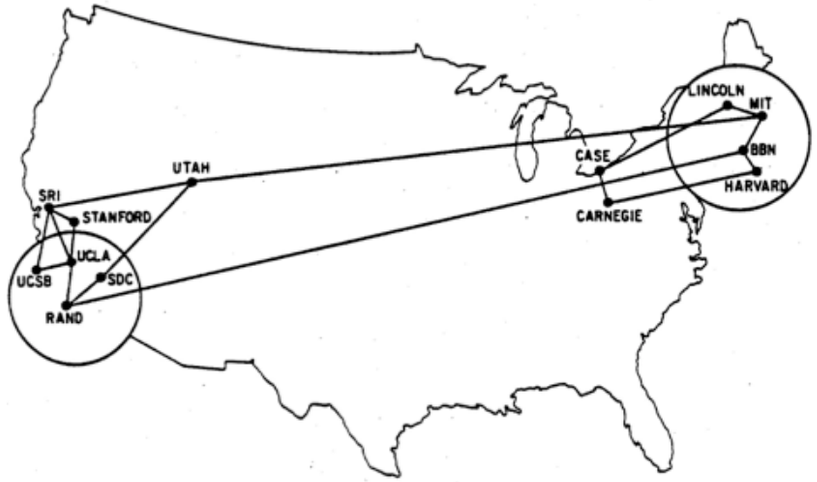
Discussion: Why is DFS usually a poor choice for finding the shortest path between two nodes (e.g., California to Massachusetts)?

Speaker notes

This slide visualizes the fundamental difference between the "layered" BFS view and the "deep" DFS view on the same infrastructure. DFS does not guarantee shortest paths (distance = 1 in BFS), as it might take 10 hops to reach a neighbor that was actually adjacent to the start.

DFS on the ARPANET





DFS applied to the ARPANET. Node numbers indicate the discovery order if neighbors are processed in the listed adjacency-list order. Notice how the search can wander far across the network before exploring nodes physically close to the start.

Discussion: Why is DFS usually a poor choice for finding the shortest path between two nodes (e.g., California to Massachusetts)?

BFS vs. DFS Transition

Property	Breadth-First Search (BFS)	Depth-First Search (DFS)
Data Structure	Queue (FIFO)	Stack (LIFO)
Expansion	Layer by layer (Ripple)	Deep and narrow (Snake)
Shortest Path	Guaranteed (unweighted)	No guarantee
Complexity	$O(\|V\| + \|E\|)$	$O(\|V\| + \|E\|)$

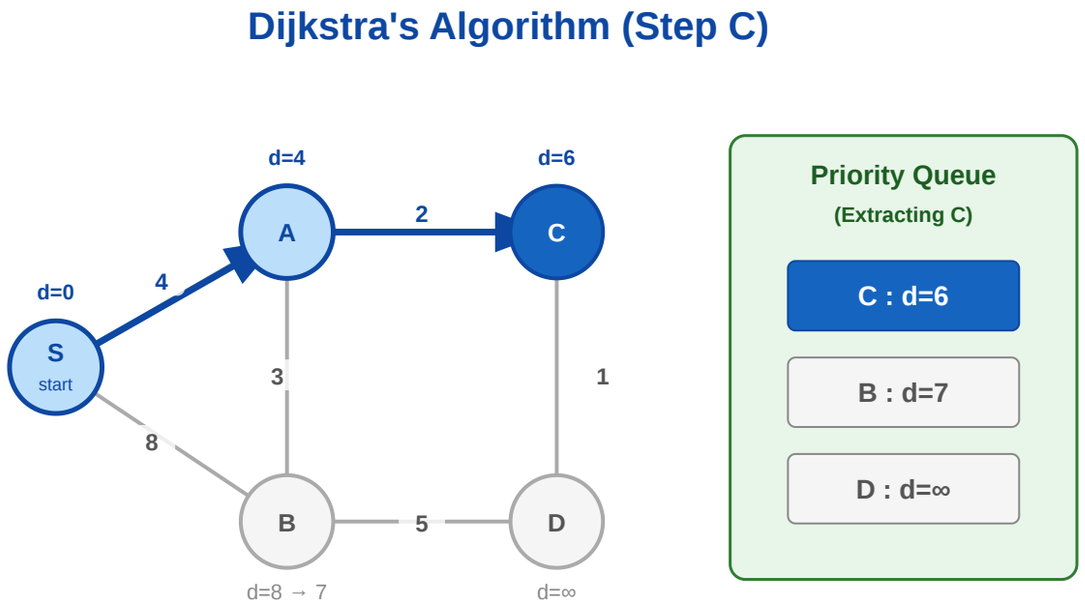
Dijkstra's Algorithm

BFS explores layer by layer. For **weighted graphs with non-negative additive edge costs**, we use Dijkstra's algorithm.

Algorithm: Dijkstra *(Input: Graph $G = (V, E)$, source s, additive weights $w \geq 0$)*

1. Set distance to s as 0 and all others as ∞.
2. Initialize a **priority queue** PQ with all nodes.
3. While PQ is not empty:

- Extract node u with the minimum distance.
- For each neighbor v of u:
  - If path through u is shorter: update v's distance.



BFS vs. Dijkstra. BFS explores identical layers. Dijkstra uses a priority queue to always expand the currently cheapest unvisited node. Complexity: $O((|V| + |E|) \log |V|)$.

Priority Queue (Min-Heap). A data structure that allows inserting elements and efficiently extracting the element with the *minimum* value. In Dijkstra, it acts as a smart queue: instead of First-In-First-Out, the node with the shortest accumulated distance is always extracted next.

Important restriction. Dijkstra assumes path costs are **additive** and every edge weight is **non-negative**. With negative weights, a path that looked final can later become cheaper, so the greedy rule breaks.

Speaker notes

Dijkstra's algorithm addresses the limitation of BFS in weighted graphs by replacing the standard queue with a priority queue (min-heap). Instead of exploring layer by layer, it greedily expands the currently "cheapest" node. This ensures that once a node is extracted from the priority queue, the shortest possible route to it has been found. This invariant requires all edge weights to be non-negative.

Quick Check

Consider our 5-node graph: `adj = {1: [2, 3], 2: [1, 4], 3: [1, 4], 4: [2, 3, 5], 5: [4]}`

Here is a BFS function to calculate the shortest path *distance* from `start` to all reachable nodes.
What goes in the two blanks?

```
from collections import deque

def bfs(graph, start):
    distances = {start: 0}
    queue = deque([start])
    while queue:
        current = queue.popleft()
        for neighbor in graph.get(current, []):
            if neighbor not in distances:
                distances[neighbor] = ________________
                queue.________(neighbor)
    return distances
```



Speaker notes

Summary of the BFS implementation details. Correct distance tracking requires updating the neighbor's distance based on the current node's value (`dist[v] = dist[u] + 1`) and ensuring that nodes are only added to the queue once to maintain the $O(|V| + |E|)$ complexity.

Quick Check — Solution

```python
from collections import deque

def bfs(graph, start):
    distances = {start: 0}
    queue = deque([start])
    while queue:
        current = queue.popleft()
        for neighbor in graph.get(current, []):
            if neighbor not in distances:
                distances[neighbor] = distances[current] + 1
                queue.append(neighbor)
    return distances
```

Key insight: Because the queue is FIFO, we always finish layer k before starting layer $k+1$. The first time we see a node, we have found its shortest path.

Speaker notes

The structural choice between a queue (FIFO) and a stack (LIFO) is the fundamental pivot between BFS and DFS. This small programming change completely alters the search strategy: BFS becomes a global shortest-path algorithm, while DFS becomes a deep path-finding and cycle-detection tool.

Summary: Graph Search

Algorithm	Structure	Strategy	Guarantees
BFS	Queue (FIFO)	Layer-by-layer	Shortest path (unweighted)
DFS	Stack (LIFO)	Deepest-first	Reachability, Cycles
Dijkstra	Priority Q	Cheapest-first	Shortest path (weighted, non-negative additive costs)

Key Insight: BFS matters because a simple local rule produces exact shortest-path information in unweighted graphs. Dijkstra extends this logic to weighted networks using a priority queue.

Speaker notes

This summary stabilizes the "Graph Search" module. While DFS, BFS, and Dijkstra share a common structural logic (examine a node, add its neighbors to a collection), the choice of collection—Stack, Queue, or Priority Queue—completely determines the algorithm's guarantees and behavior. Understanding this mapping is the key to algorithmic intuition in network science.

Connectivity

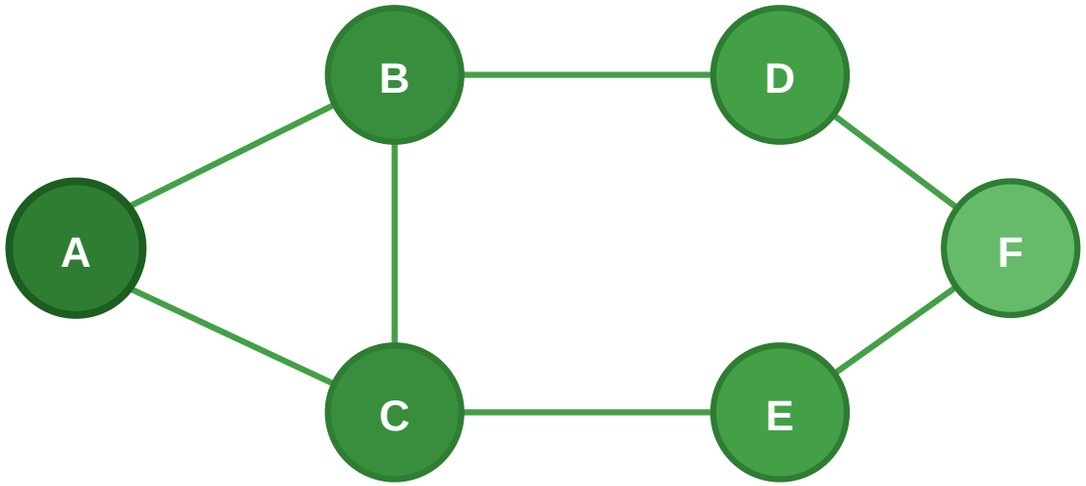
Guiding Question: When is a network still one system — and when has it split apart?

Connectedness

Connected (undirected). Let $G = (V, E)$ be a graph. G is *connected* if for every pair $u, v \in V$ there exists a path from u to v.

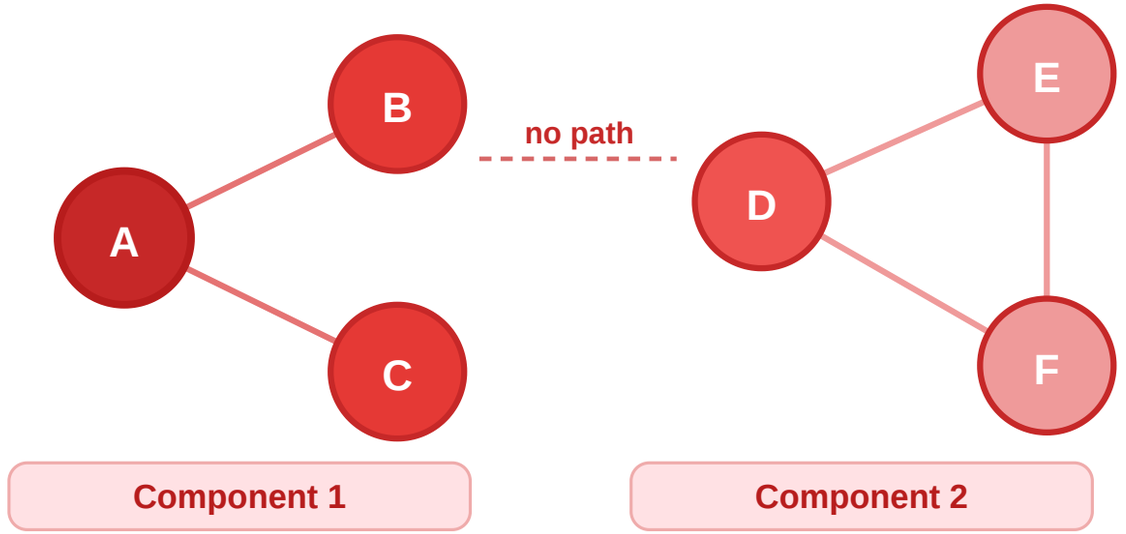
Connected Graph

A path exists between every pair of nodes



Disconnected Graph

No path exists between the two clusters



Speaker notes

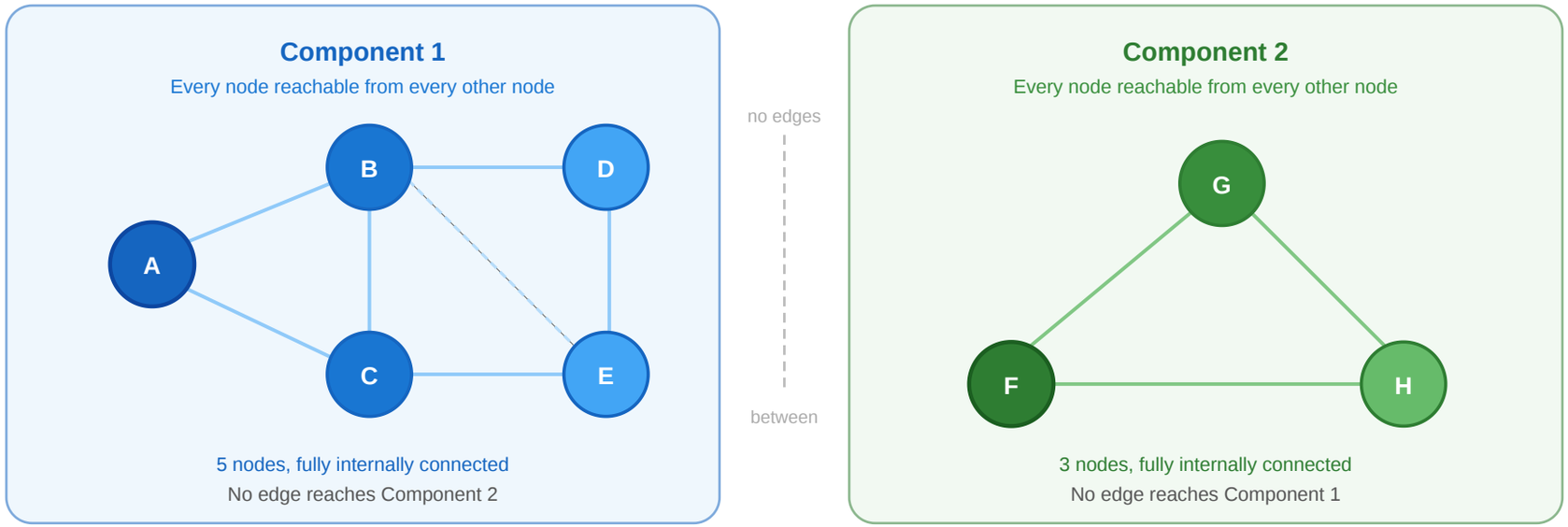
Connectedness is a global property of a graph: if a path exists between every pair of nodes, the system is a single entity. Disconnectedness indicates fragmentation, where the network has "islands" that cannot communicate or influence each other. This distinction is critical for understanding the reach of diffusion processes.

Connected Components

Connected components are maximal connected subgraphs.

Connected component. A *connected component* of G is a subgraph $H \subseteq G$ such that:

1. H is **connected** (every pair of vertices in H has a path within H), and
2. H is **maximal** — no vertex outside H has a path to any vertex in H.



Each colored cluster is a maximal connected subgraph — meaning it cannot be extended without losing separation.

Typical examples:

- separate friendship groups if no acquaintance links the groups
- islands in a road network if no bridge or ferry connects them
- disconnected subnetworks in technical infrastructure after a link failure

Speaker notes

Connected components are formally defined by two properties: connectivity and maximality. A component must be fully reachable internally, but it must also include every possible node—if an external node has a path to the component, that node should have been part of the component itself. Identification of these maximal sets is the first step in analyzing the large-scale structure of fragmented networks.

Connectedness in Directed Graphs

How does connectivity change when edges have direction?

Speaker notes

In directed graphs, connectivity becomes more nuanced. Weak connectivity ignores edge direction, while strong connectivity requires paths to be traversable in both directions. This distinction is essential for modeling the Web: a page might be reachable from the "giant component" but unable to link back to it, placing it in a separate strongly connected component despite being part of the same weak structure.

Connectedness in Directed Graphs

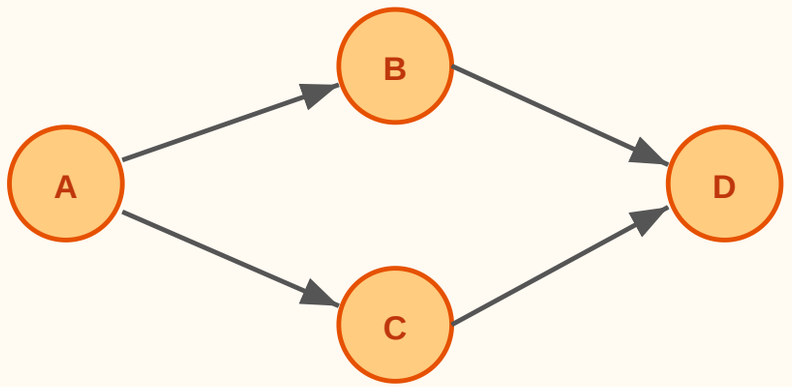
How does connectivity change when edges have direction?

Strongly connected. For every pair $u, v \in V$ there is a directed path from u to v **and** from v to u.

Weakly connected. The graph is connected when all directed edges are replaced with undirected edges.

Weakly Connected

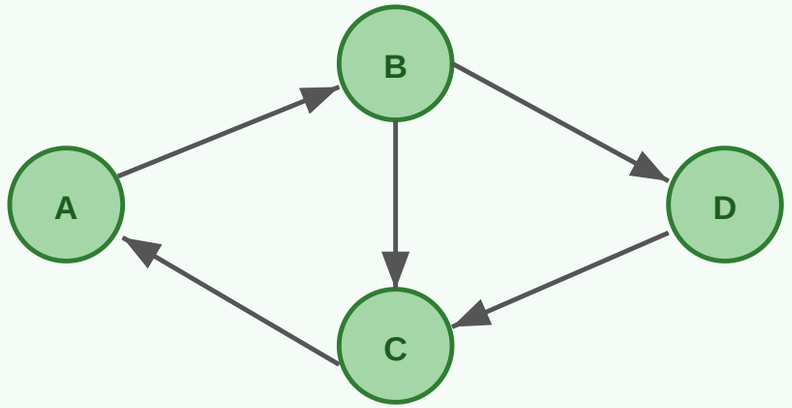
(directed, NOT strongly connected)



A can reach D, but D cannot reach A

Strongly Connected

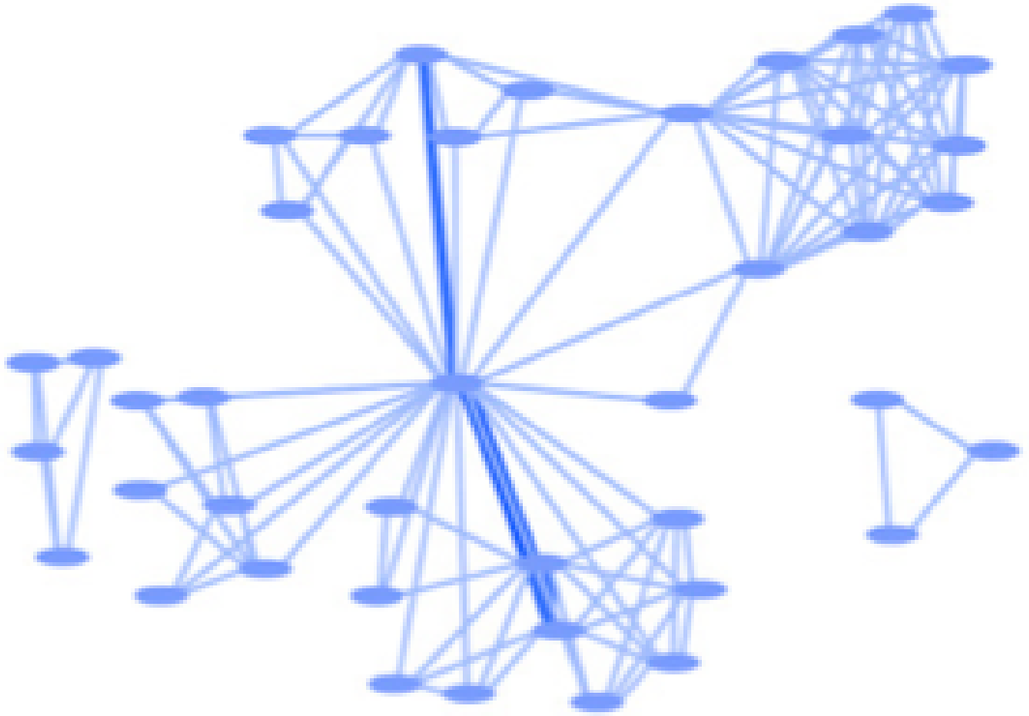
(directed, MUTUALLY reachable)



Every node can reach every other node

Real-World Component Patterns

Many real networks contain one large giant component and several much smaller disconnected parts. Below is a collaboration network with one giant component plus several small peripheral groups.



Source: Easley & Kleinberg 2010

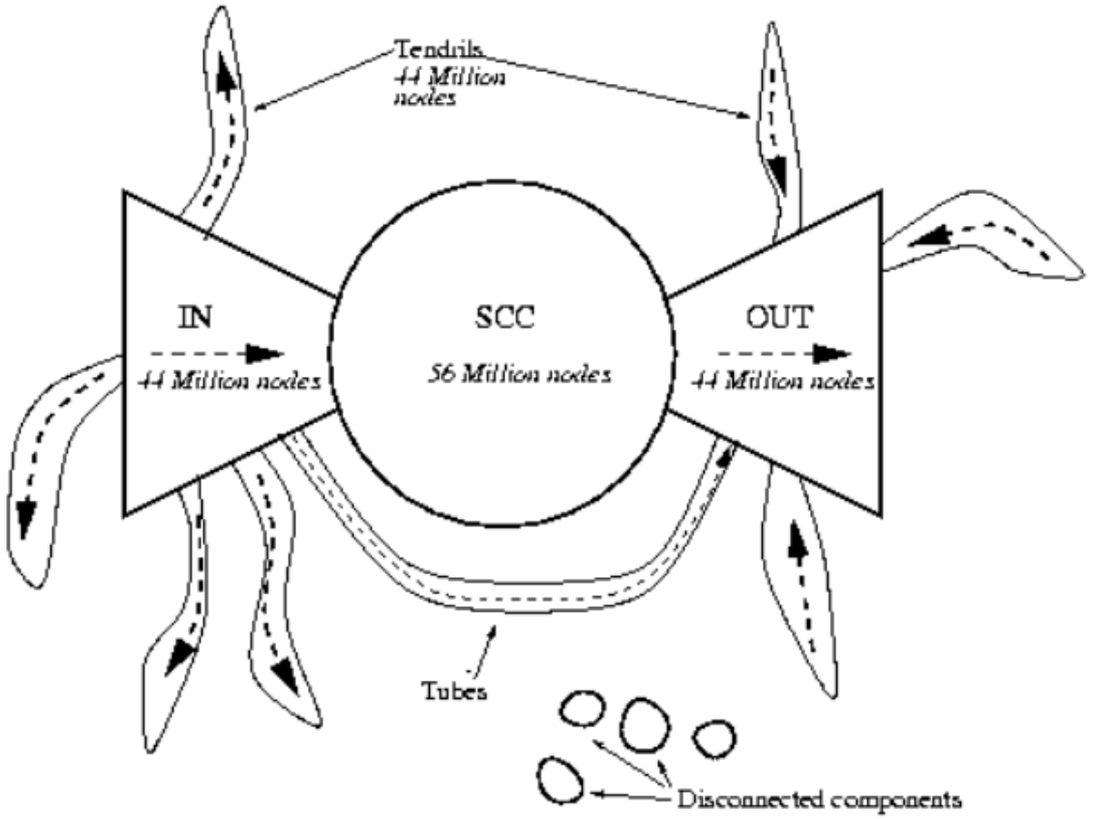
Giant components are quite common:

- **Co-authorship networks:** one giant component plus a few isolated teams
- **Infrastructure networks:** failures can split one system into several components
- **Web graph:** one weakly connected macro-

structure, but many different strongly connected regions

Directed Components: The Web Bow Tie

The Web is the classic example that weak and strong connectivity are not the same.



Source: Easley & Kleinberg 2010, p. Ch. 13

Broder et al. (2000), *Graph structure in the Web*, *Computer Networks* 33(1-6):309-320, showed that the Web decomposes into:

- **SCC:** pages mutually reachable by directed paths
- **IN:** pages that can reach the SCC but are not reachable from it
- **OUT:** pages reachable from the SCC but unable to link back
- **Tendrils / tubes:** side regions attached to IN or OUT

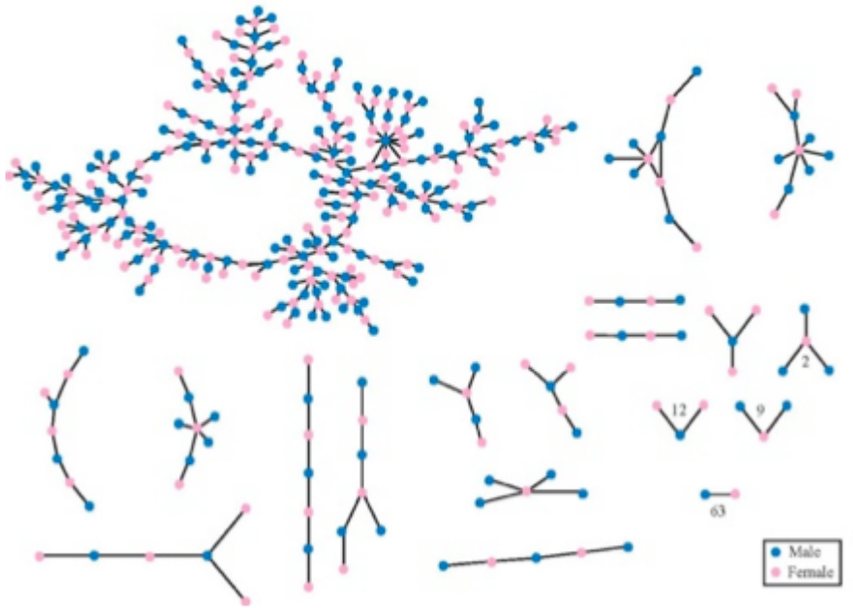
This is why directed networks need more than the undirected idea of "one big connected piece."

Speaker notes

Every weakly connected component can be decomposed into strongly connected components, but the Web makes the distinction concrete. The SCC is the part where mutual reachability holds; IN and OUT are attached only one way. Broder et al. (2000) is the canonical large-scale example of directed component structure.

Giant Components

Giant component. A connected component that is significantly larger than all others, usually containing a macroscopic fraction of all vertices. Appears quite often in real world networks.



Source: Easley & Kleinberg 2010

Takeaway

> Connected components and giant components tell us whether interaction, diffusion, or coordination can happen across the whole graph or only inside local regions.

Speaker notes

The study of connectivity culminates in themes of robustness and fragmentation. Whether a network remains cohesive or splits into isolated components determines how effectively it can coordinate behavior or survive attacks. These structural properties set the stage for later topics such as community detection and epidemic modeling.

Quick Check

We can use BFS to find all connected components by repeatedly starting search from unvisited nodes. **What goes in the blanks?**

```
def find_components(graph):
    visited = set()
    components = []
    for node in graph:
        if node not in visited:
            # bfs returns {node: distance} for all reachable nodes
            reachable = bfs(graph, node)
            component = set(reachable.keys())

            # Update global visited set and store result
            visited |= __________
            components.append(__________)
    return components
```

Discussion Points:

- How does BFS naturally identify a "maximal" component?
- For directed graphs, does this find SCCs or WCCs?

Speaker notes

Connected component detection is a direct application of BFS. By running BFS repeatedly from unvisited nodes, we can systematically bucket every node into its maximal connected set. This demonstrates the algorithmic power of simple graph search for global structural discovery.

Quick Check — Solution

Strategy: Start from an unvisited node, run BFS to find all reachable nodes. Repeat until all nodes visited.

```python
def find_components(graph):
    visited = set()
    components = []
    for node in graph:
        if node not in visited:
            reachable = bfs(graph, node)
            component = set(reachable.keys())

            # Solution:
            visited |= component
            components.append(component)
    return components

# Test with disconnected graph
graph = {
    "A": ["B"], "B": ["A", "C"], "C": ["B"],   # component 1
    "X": ["Y"], "Y": ["X", "Z"], "Z": ["Y"]    # component 2
}
print(find_components(graph))
# [{'A', 'B', 'C'}, {'X', 'Y', 'Z'}]
```

Speaker notes

Identifying components is computationally straightforward for undirected graphs using BFS or DFS. However, directed graphs require more sophisticated algorithms to account for asymmetric reachability. This module concludes the structural foundations, providing the tools needed to move from individual paths to global network architecture.

Wrap-Up

- **Networks have properties** that demand formal description
- **Graphs** provide a precise model: nodes, edges, weights, direction, time
- **Graph variants** can be labelled, typed, attributed, sparse, dense, or random
- **Paths** give us reachability and distance
- **BFS** is a central traversal algorithm with guaranteed shortest paths
- **Dijkstra's** extends BFS to weighted graphs with non-negative additive costs
- **Connectivity** helps us detect structure and giant components

Reading

Chapter 2 of Easley & Kleinberg (2010) covers graph theory foundations, paths, BFS, and connectivity in detail.

Speaker notes

This lecture has established the fundamental vocabulary of network science. By translating intuitive connections into formal graphs, we have moved from "knowing" a network exists to being able to precisely describe its properties, calculate its distances, and analyze its connectivity. These foundations are the prerequisite for every subsequent topic in the course, from social influence to web search.

Frequently Asked Questions (FAQ)

Why would we use DFS in practice?

- to test **reachability**: is there any path at all from a start node to a target, even if we do not care about the shortest one?
- to detect **cycles**: if DFS encounters a node that is already on the current recursion stack, we have found a loop
- to build **topological orderings** in DAG-like settings: finishing times from DFS give an ordering that respects dependencies
- to explore one branch deeply before trying alternatives

DFS is not the shortest-path tool. It is the structural exploration tool.

Why is DFS useful for connected component detection?

Because one DFS from a start node visits **exactly the nodes reachable from that node**. In an undirected graph, that reachable set is one full connected component. Repeating DFS from each still-unvisited node therefore partitions the graph into components.

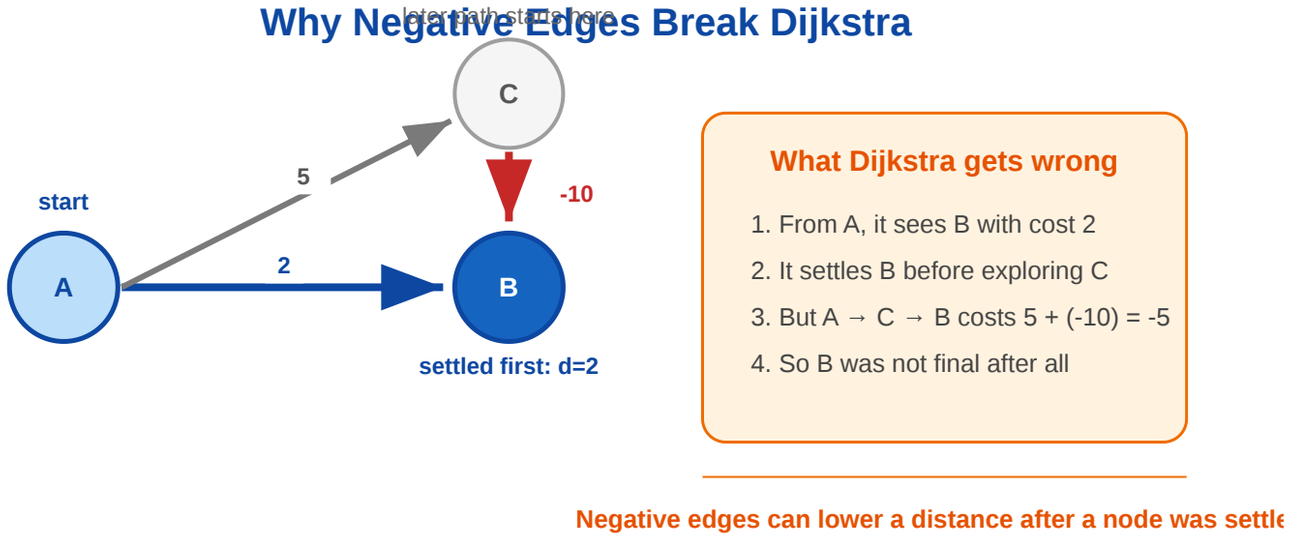
DFS is also a common practical choice because the implementation is simple: we only need a visited set plus recursion or an explicit stack. BFS works just as well for component detection, but it is not generally true that DFS always uses less memory; that depends on the graph shape.

Why do negative weights break Dijkstra?

Dijkstra assumes that once the currently cheapest node is removed from the priority queue, its distance is final. That is only true when every extra edge can only **increase** total cost.

With a negative edge, a later detour can make an already-settled node cheaper after all:

```
A -> B = 2, A -> C = 5, C -> B = -10
```



At first, Dijkstra would settle `B` with cost `2`. But the path `A -> C -> B` has total cost `-5`, so the earlier decision was wrong.

Image Credits & References

Easley, David and Kleinberg, Jon (2010). Networks, Crowds, and Markets: Reasoning about a Highly Connected World. *Cambridge University Press*. https://www.cs.cornell.edu/home/kleinber/networks-book/